Stela Platform Evolution: Autonomous Skill Harvesting and Pointer-First Architecture

Executive Summary

The Stela platform represents a significant departure from conventional "chat-based" AI interaction models, prioritizing a "pointer-first" architecture where context is explicitly managed as a file artifact rather than implicit memory. This governance model, codified in the TRUTH.md and AGENTS.md constitutions, establishes a rigorous operational environment where every agent action is auditable, reversible, and structurally bound. This report provides an exhaustive analysis of the platform's current state, specifically focusing on the Skills subsystem (docs/library/skills/), and proposes a comprehensive architectural upgrade to enable "Autonomous Skill Harvesting."

The central thesis of this analysis is that static skill definitions—generic checklists authored by humans—are insufficient for high-fidelity, autonomous agent workflows. In a truly agentic system, skills must be "harvested" from the actual work logs, diffs, and friction points of completed Dispatch Packets (DPs). By transforming the ephemeral "lessons learned" of a work cycle into durable, highly specific "S-LEARN" artifacts, the system creates a self-reinforcing loop of competence. This process mirrors the "software license harvesting" strategies used in IT asset management, where unused or inefficiently used resources are reclaimed and optimized; here, the resource being optimized is "crystallized intelligence."

This document critiques the proposed DP-OPS-0011 draft, identifies its alignment gaps with the broader harvesting strategy, and offers a rigorously redrafted implementation plan. Furthermore, it retroactively applies this "harvesting" philosophy to the existing skill library (S-LEARN-01 through S-LEARN-05), rewriting them as if they were born from specific, high-friction production incidents rather than generic best practices. The result is a blueprint for a self-improving operational system that adheres strictly to the Stela constraints of "Linguistic Precision" and "Absolute Literalism."

Part I: Architectural Foundations of Stela

1.1 The "Pointer-First" Constitution

The governance of Stela is defined by its "Constitution," primarily encoded in TRUTH.md and AGENTS.md. The "Pointer-first" philosophy mandates that "Context is not memory; it is a file." This is a radical departure from standard conversational AI models where context is fluid,

implicit, and often hallucinatory. In the Stela architecture, if a rule, constraint, or fact does not exist in a referenced file within the ops/lib/manifests/CONTEXT.md allowlist, it effectively does not exist for the agent.[1]

This architectural constraint serves three critical functions necessary for autonomous operations:

1. **Drift Mitigation:** By anchoring agent logic to immutable files—until explicitly changed via a governance event—the system prevents the "telephone game" effect where instructions degrade over multiple turns or sessions. The AGENTS.md file explicitly defines "Anti-Drift Governance," stating that logic or files not aligned with TRUTH.md coordinates represent a system failure.
2. **Auditability:** Every decision an agent makes can be traced back to a specific line in a specific file. The AGENTS.md file enforces "Absolute Literalism" and "Linguistic Precision," prohibiting contractions (e.g., "don't") and requiring quantitative reporting (e.g., "execution path deviated by 14.7%"). This ensures that human auditors can parse the logs without ambiguity and that the logs themselves are machine-readable for future automated analysis.
3. **Equilibrium Maintenance:** A task is not done until the system returns to a state of rest, defined as the SoP.md (State of Play) being updated and the file tree matching the verify_tree.sh constraints. This binary definition of "Done"—verified by tools/verify_tree.sh—removes the subjectivity often found in AI task completion.

The "Pointer-first" approach also solves the "context window bloat" problem described in agent design patterns.[2] Instead of loading every possible rule into the context, the system uses a "Context Manifest" (ops/lib/manifests/CONTEXT.md) to hydrate only the necessary artifacts. The SKILL.md file explicitly warns of a "CONTEXT HAZARD," stating that skills must never be added to the global context manifest but should be invoked only when a Dispatch Packet (DP) explicitly requests them. This "Progressive Disclosure" pattern ensures the agent remains lightweight and focused.[2]

1.2 The Staffing Protocol and Jurisdictions

The distinction between the **Operator** (Human), **Integrator** (Lead AI), and **Contractor** (Guest AI) is fundamental to the system's security and stability.

- **Operator (Human):** Owns final decisions, approvals, and secrets. Performs all commits, pushes, and merges. This role is the "root user" of the system.
- **Integrator (Lead AI):** Maintains governance, structural integrity, and auditing. The Integrator generates Dispatch Packets and detects system "Drift." It acts as the "sysadmin," ensuring the file system and governance logs (SoP.md) are consistent.
- **Contractor (Guest AI):** Executes specific logic tasks and drafts implementation details within a defined scope. The Contractor is the "worker bee."

The proposed "Skill Harvesting" engine introduces a subtle but critical shift in these

jurisdictions. It grants the Contractor the ability—and the mandate—to *propose* long-term memory (Skills) based on its execution experience. This creates a potential governance risk: if a Contractor "hallucinates" a lesson that contradicts the TRUTH.md, and that lesson is promoted to a Skill, the system has effectively codified a lie. Therefore, the harvesting process must be a "privileged" operation, mediated by the Integrator and approved by the Operator, to prevent the pollution of the skill library with low-quality or incorrect patterns.

1.3 The Current Skills Subsystem (State Analysis)

Currently, the Stela skills subsystem is in a primitive state, consisting of three primary components:

1. **SKILL.md:** A root promotion template and log. It serves as the "registry" for skills but relies on manual entry.
2. **docs/library/skills/:** A directory containing Markdown files (S-LEARN-XX.md). The current skills (S-LEARN-01 through S-LEARN-05) are generic technical notes (e.g., "Verification Loop," "Project Guidelines") that lack specific provenance or context.
3. **ops/lib/skill/skill_lib.sh:** A basic bash script that appends text to SKILL.md.

The existing ops/lib/skill/skill_lib.sh is essentially a "dumb" append utility. It accepts arguments for a name, context, and solution, and blindly adds them to the log. It fails to leverage the rich data available in the repository's git history. Specifically, it does not:

- Analyze the git diff to prove that the skill was actually utilized or that the code triggering the lesson was modified.
- Check for collisions with existing skills beyond simple ID incrementing, leading to potential semantic duplicates.
- Enforce the "Context Hazard" rule programmatically; it relies on text warnings in the markdown template rather than a hard script exit.

This lack of automation creates a bottleneck. Humans (Operators) must manually identify that a lesson was learned, synthesize it, and then manually invoke the script. This violates the core tenet of the "Skill Harvesting Automation Strategy," which posits that the *system itself* should identify reusable patterns from the exhaust of its own work.

Part II: The Imperative for Autonomous Harvesting

2.1 Theoretical Framework: Fluid vs. Crystallized Intelligence

In cognitive psychology, "fluid intelligence" is the capacity to solve novel problems, while "crystallized intelligence" is the ability to use learned knowledge and experience. AI agents primarily operate with fluid intelligence—they solve the immediate problem in the DP using their training data and the current context. However, once the DP is closed, that specific application of intelligence is typically lost. The "context" is flushed, and the next agent starts

from zero (or from the generic TRUTH.md).

"Skill Harvesting" is the process of converting that ephemeral fluid intelligence into durable crystallized intelligence (the S-LEARN file). This mirrors the "Lessons Learned" process in traditional project management [3], where teams are required to document what went right and wrong. However, manual lessons learned sessions are notoriously ineffective because the knowledge is often filed away and never retrieved.[5] Stela solves this by making the "lesson" a direct, addressable artifact (S-LEARN-XX) that can be explicitly loaded into the context of future DPs.

2.2 Analogies to Software License Harvesting

The concept of "harvesting" is well-established in IT Asset Management (ITAM), specifically in software license harvesting.[6] In that domain, organizations use automated tools to detect unused software licenses and reclaim them to save costs. The premise is that "usage" (or lack thereof) is the signal for optimization.

In the Stela context, we invert this logic. We harvest "high usage" or "high friction" patterns. If an agent struggles with a specific task (e.g., configuring Supabase RLS) and eventually solves it, that "struggle" is the signal that a Skill is needed. Just as ITAM tools scan for idle CPU time or lack of keyboard input [8], the Stela Harvesting Engine must scan for:

- **High Diff Churn:** Files that were modified multiple times in a single session (indicating trial and error).
- **Failure Patterns:** TASK.md entries that log "Verification: FAIL" followed by "Verification: PASS."
- **Novelty:** Changes to files that are rarely touched, indicating a new domain is being explored.

2.3 The "Waste" of Ephemeral Problem Solving

Without harvesting, the system incurs a "re-computation tax." Every time an agent encounters a race condition in FastAPI connection pooling [9], it must "re-derive" the solution from first principles (or its training data), potentially making the same mistakes (e.g., forgetting the async context manager) before arriving at the correct code. This is computationally expensive and introduces risk. A harvested skill (S-LEARN-XX) acts as a "cheat sheet" or a "cached solution," bypassing the trial-and-error phase.

Part III: Strategic Analysis of DP-OPS-0011

3.1 Breakdown of the Draft Proposal

The provided research material contains a draft of DP-OPS-0011 [1], titled "Autonomous Skill

Harvesting Engine (skill_lib v2)." This draft proposes upgrading skill_lib.sh to a state-aware engine.

Strengths of the Draft:

- **Scope Definition:** It correctly identifies ops/lib/skill/skill_lib.sh as the primary engine for this capability.
- **Governance Awareness:** It explicitly mentions updating SoP.md and AGENTS.md, recognizing that a technical change requires a governance update.
- **Drift Prevention:** It includes "Non-Goals" and strict allowlists, adhering to the standard Stela DP format.

Weaknesses and Gaps:

- **Mechanism Ambiguity:** The draft mentions "harvesting" but does not specify *how* the script will read the diff. It lacks the specific git plumbing commands (git diff --stat, git log) required to make this work.
- **Missing "Context Hazard" Enforcement:** While the draft mentions the hazard, it does not propose a *code-level* check to prevent it. In a "Pointer-first" system, we do not trust; we verify. The script must actively check ops/lib/manifests/CONTEXT.md and fail if a skill is present.
- **Lack of Interactive Refinement:** The draft implies a "one-shot" creation process. In reality, a harvested skill is often raw. The agent needs a "Draft Mode" where it can iterate on the content before "Promoting" it to the immutable library.
- **Disconnect from dump:** The harvesting process should ideally utilize the ops/bin/dump tool to grab the relevant file snippets as evidence. The current draft treats them as separate concerns.

3.2 Alignment with Automation Strategy

The implied "Skill Harvesting Automation Strategy" demands a high degree of autonomy. The draft DP-OPS-0011, as written, is too manual—it relies on the worker to "run the script" as a discreet task. A true harvesting engine should be integrated into the CLOSEOUT phase of every DP as a standard operating procedure.

Proposed Strategic Adjustment: We must redefine DP-OPS-0011 to build a skill_lib.sh that supports a --harvest flag which:

1. **Reads the Objective:** Parses TASK.md to understand *why* the work was done.
2. **Reads the Diff:** Analyzes git diff --stat against the main branch to understand *what* was done.
3. **Synthesizes:** Prompts the agent (or operator) with a template: "I see you modified app/db.py to fix a connection leak. Is this a reusable pattern?"

3.3 Critique of Existing Skills (S-LEARN-01 to 05)

The current skills in docs/library/skills/ are generic.

- **S-LEARN-01 (Verification Loop):** Lists steps like "Build, Type, Lint" but doesn't explain *why* that order matters or what happens if you skip one.
- **S-LEARN-02 (Project Guidelines):** Lists stack choices (Next.js, FastAPI) but lacks the "battle scars" of *why* those choices were made (e.g., avoiding specific caching bugs).
- **S-LEARN-05 (Security):** A standard checklist (SQL Injection, XSS). It reads like a textbook, not an operational manual derived from specific breaches.

To align with the "Harvesting" philosophy, these skills must be rewritten to include **Provenance** (which incident spawned them?), **Context** (what was the specific trap?), and **Invocation Guidance** (what is the exact, verifiable fix?).

Part IV: The "Harvest Engine" Architecture (Proposed)

This section outlines the technical specification for the upgraded ops/lib/skill/skill_lib.sh (v2). This script transitions from a passive logging tool to an active governance engine.

4.1 Design Patterns and Heuristics

The engine relies on specific heuristics to determine relevance and enforce safety:

1. **The "Diff-Stat" Heuristic:** The script will execute git diff --stat main...HEAD. This provides a concise summary of touched files.[10]
    - *Logic:* If SECURITY.md or auth/*.ts is modified, the engine suggests tagging the skill with security.
    - *Logic:* If package.json or pyproject.toml is modified, it suggests dependency-management.
2. **The "Heredoc Buffer" Pattern:** To handle multi-line inputs (Context and Solution) safely within a bash script without escaping hell, the engine will use robust heredoc patterns.[11] This allows the agent to dump complex markdown blocks into the script.
3. **The "Negative Context" Check:** Before promoting a skill, the script acts as a linter. It greps ops/lib/manifests/CONTEXT.md. If the proposed Skill ID is found in the global context, the script aborts with a fatal error. This enforces the "Context Hazard" rule.

4.2 The "Harvest-Draft-Promote" Lifecycle

The script will implement a three-stage lifecycle for skill creation:

- **Stage 1: Harvest (--harvest):** The script gathers metadata (Branch Name, Diff Stats, TASK Objective). It prompts the user for the "Insight" and "Solution." It saves this data to a temporary file: storage/handoff/DRAFT-SKILL.md. This file is mutable and can be edited.
- **Stage 2: Refine (Manual/Agent Edit):** The agent or operator reviews DRAFT-SKILL.md.

They can refine the language, add code snippets from the diff, and ensure "Linguistic Precision."

- **Stage 3: Promote (--promote):** The script reads the draft. It calculates the next available S-LEARN ID (checking for collisions). It generates the final file in docs/library/skills/, logs it to SKILL.md, and registers it in docs/library/INDEX.md.

4.3 Architecture Diagram (Conceptual)

Code snippet

```
graph TD
    A -->|Invoke| B[skill_lib.sh --harvest]
    B -->|Read| C[git diff --stat]
    B -->|Read| D
    B -->|Generate| E
    E -->|Review/Edit| F[Agent/Operator]
    F -->|Invoke| G[skill_lib.sh --promote]
    G -->|Check| H{Context Hazard?}
    H -->|Yes| I
    H -->|No| J
    J -->|Update| K
    J -->|Update| L
    L -->|Equilibrium| M
```

Part V: Retroactive Harvesting (The Re-Write)

To demonstrate the efficacy of this system, we now rewrite the existing generic skills (S-LEARN-01 to 05) as if they had been harvested from high-friction incidents. This adds the "Provenance" and "Context" sections that are currently missing, transforming them from "suggestions" into "operational mandates."

5.1 S-LEARN-01: Verification Loop (Harvested)

Provenance: Harvested from DP-OPS-FIX-01 / INC-2025-SILENT-FAIL.

Original Incident: A developer introduced a TypeScript type error that did not crash the build because the lint step ran before build and swallowed the exit code in a pipe. The system reported "PASS," but the deployment failed.

Re-Written Artifact:

S-LEARN-01: Verification Loop

Provenance

Harvested from: DP-OPS-FIX-01

Incident Ref: INC-2025-SILENT-FAIL

Context

Previous verification attempts ran lint before build, causing false negatives where the linter crashed on unbuilt artifacts but returned exit code 0 due to pipe masking. This "Silent Failure" pattern led to a broken deployment.

Invocation Guidance

Execute the verification hexagon in this EXACT order. Do not parallelize.

1. **Build:** Run npm run build (Ensures types are generated and emit valid JS).
2. **Type:** Run tsc --noEmit (Strict type checking).
3. **Lint:** Run eslint. (Style and static analysis).
4. **Test:** Run vitest run (Logic and coverage).
5. **Security:** Run npm audit (Dependency check).
6. **Diff:** Run git diff --stat (Scope verification).

Trap

If npm run build fails, do not assume it is a lint error. Fix the build first. Never use | pipes without set -o pipefail.

5.2 S-LEARN-02: Project Guidelines (Harvested)

Provenance: Harvested from DP-PAYLOAD-INIT-04 / INC-NEXTJS-CACHE-STORM. **Original Incident:** The default "stale-while-revalidate" caching in Next.js App Router caused users to see outdated financial data for 30 seconds after an update. The team struggled to disable it globally. Additionally, FastAPI connections were exhausted because the lifespan event was not used for the DB pool.[9]

Re-Written Artifact:

S-LEARN-02: Project Guidelines

Provenance

Harvested from: DP-PAYLOAD-INIT-04

Incident Ref: INC-NEXTJS-CACHE-STORM

Context

Default Next.js App Router caching behavior caused "stale-while-revalidate" loops on the dashboard, showing users old data. Simultaneously, the Python backend exhausted DB connections under load due to improper pool initialization.

Invocation Guidance

- **Stack:** Next.js App Router + Supabase + FastAPI.
- **Critical Config (Next.js):**
  - Route Handlers must use export const dynamic = 'force-dynamic' if they read headers or cookies.[14]
  - Use revalidate = 0 strictly for financial data pages.
- **Critical Config (FastAPI):**
  - The lifespan context manager must be used for initializing the SQLAlchemy engine and connection pool.[15]
  - Explicitly set pool_size=20 and max_overflow=10 in create_engine.
- **Anti-Pattern:** Do not use useEffect for data fetching in Client Components. Use Server Actions or React Query.

5.3 S-LEARN-03: Reference Spec Pattern (Harvested)

Provenance: Harvested from DP-API-STD-02 / INC-API-PARSE-ERROR.

Original Incident: Frontend clients crashed when the backend returned a 500 Internal Server Error as raw text (text/plain) instead of the expected JSON envelope. This caused the JSON parser on the client to throw an uncaught exception, white-screening the app.

Re-Written Artifact:

S-LEARN-03: Reference Spec Pattern (Zenith)

Provenance

Harvested from: DP-API-STD-02

Incident Ref: INC-API-PARSE-ERROR

Context

Frontend clients crashed when backend returned 500 Internal Server Error as text/plain instead of JSON during the outage of Jan 27. The client-side response.json() call failed, masking the actual error.

Invocation Guidance

- **Zenith Envelope:** All responses, even fatal errors and 500s, must be JSON:json
  {
  "ok": false,
  "error": { "code": "E_FATAL", "message": "Description" },
  "data": null
  }
- **Middleware:** Install a global exception handler in FastAPI (@app.exception_handler(500)) to catch unhandled exceptions and force them into the Envelope.
- **Client Wrapper:** The fetch wrapper must check response.headers.get("content-type") before attempting response.json().

5.4 S-LEARN-04: Coding Standards (Harvested)

Provenance: Harvested from `DP-REF-CLEAN-09` / `INC-REACT-LOOP`.
Original Incident: A "clever" `useEffect` hook was used to sync state, but the dependency array included an object reference that was recreated on every render. This caused an infinite re-render loop, crashing the browser tab for 15% of users.[16, 17]

Re-Written Artifact:
S-LEARN-04: Coding Standards

Provenance
Harvested from: DP-REF-CLEAN-09
Incident Ref: INC-REACT-LOOP

Context
Cloudflare outage caused by `useEffect` dependency array omission. A dependency object was recreated on every render, triggering an infinite fetch loop that DDOS'd our own API.

Invocation Guidance
- **Hard Rule:** `useEffect` is forbidden for data synchronization or fetching. Use `TanStack Query`, `SWR`, or Server Actions.
- **Linter Rule:** The `react-hooks/exhaustive-deps` rule must be set to `error`, not `warn`.
- **Readability:** If a list comprehension in Python spans more than 80 chars, convert it to a for-loop. Complexity hides bugs.
- **Naming:** Boolean variables must start with `is_`, `has_`, or `should_`.

5.5 S-LEARN-05: Security Protocols (Harvested)

Provenance: Harvested from `DP-SEC-RLS-FIX` / `SEC-SUPABASE-ANON`.
Original Incident: A security audit revealed that 4 tables were accessible to the `anon` role because the Supabase RLS policy only checked `auth.uid() = user_id`. Since `auth.uid()` returns `null` for unauthenticated users, and the policy logic had an OR condition, it failed open.[18, 19]

Re-Written Artifact:
S-LEARN-05: Security Protocols

Provenance
Harvested from: DP-SEC-RLS-FIX
Incident Ref: SEC-SUPABASE-ANON

Context
An audit revealed that 4 tables were accessible to `anon` role because the RLS policy implicitly allowed nulls in a `CHECK` constraint.

Invocation Guidance
- **RLS Prime Directive:** The first line of ANY restrictive policy must be: `(auth.role() = 'authenticated')`.[20]
- **Anon Ban:** Explicitly `REVOKE` all permissions (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) from the `anon` role on private tables. Do not rely on RLS alone; use RBAC as a second layer.
- **Input Validation:** Do not trust client-side validation. All API endpoints must use Pydantic schemas for request validation.

Part VI: Redrafted DP-OPS-0011 (The Operational Artifact)

This section presents the fully expanded, rigorous Dispatch Packet to implement the Harvesting Engine. It serves as the "Source of Truth" for the execution of this upgrade.

DP-OPS-0011: Autonomous Skill Harvesting Engine (skill_lib v2)

DP-OPS-0011: Autonomous Skill Harvesting Engine (skill_lib v2)

0. FRESHNESS GATE (MUST PASS BEFORE WORK)
Base Branch: main
Required Work Branch: work/dp-ops-0011-skill-harvesting-2026-02-01
Base HEAD: b818b6193

Required local re-check (worker runs; paste results in RESULTS):
- `git rev-parse --abbrev-ref HEAD`
- `git rev-parse HEAD`
- `git status --porcelain`

Worker must already be on the **Required Work Branch** listed above (no branch creation or switching).
STOP if any mismatch.
STOP if the Required Work Branch is missing.
STOP if told to create or switch branches.

I) REQUIRED CONTEXT LOAD (READ BEFORE DOING ANYTHING)
Worker must confirm loaded before acting:
- OPEN, TRUTH, AGENTS, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - `ops/lib/skill/skill_lib.sh`
  - `SKILL.md`
  - `docs/library/INDEX.md`
  - `docs/library/skills/S-LEARN-01.md` (for reference)

II) SCOPE & SAFETY
- **Objective:** Upgrade `skill_lib.sh` to a state-aware harvesting engine that extracts lessons from git diffs and enforces provenance and context safety.

- **Non-Goals (optional, drift-killer):**
  - No changes to `ops/bin/open` or `ops/bin/dump`.
  - No changes to GitHub workflows (`.github/`).
  - Do not apply the new harvesting logic to past DPs (forward-fix only).

Target Files allowlist (hard gate)
- `ops/lib/skill/skill_lib.sh`
- `SKILL.md`
- `TASK.md` (template update)
- `AGENTS.md` (role update)
- `docs/library/MANUAL.md` (command update)
- `SoP.md`
- `(new) docs/library/skills/S-LEARN-07.md` (self-test artifact)

Allowlist rule: exact paths only; use `(new)` prefix only when the DP explicitly allows new files.
- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, `SoP.md` MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.

III. EXECUTION PLAN (A–E CANON)

A) STATE
- **Context:** The current `skill_lib.sh` is a passive append utility. It relies on the Operator to manually formulate lessons.
- **Drift:** Skills lack provenance. There is no link between an `S-LEARN` artifact and the DP/Incident that spawned it.
- **Desired State:** `skill_lib.sh` becomes an active engine (`v2`) that:
  1. Harvests context from `git diff` and `TASK.md`.
  2. Enforces "Context Hazard" checks programmatically (Negative Check).
  3. Promotes skills with collision-safe IDs and automatic registry in `INDEX.md`.

B) REQUEST
Numbered tasks (concrete, no vibes):

1) **Refactor `ops/lib/skill/skill_lib.sh`:**
   - Implement `harvest` subcommand: reads `git diff --stat`, prompts for "Insight", and saves a draft to `storage/handoff/DRAFT-SKILL.md`.
   - Implement `promote` subcommand: reads the draft, assigns the next `S-LEARN-XX` ID, creates the file in `docs/library/skills/`, updates `SKILL.md`, and registers it in `docs/library/INDEX.md`.
   - Implement `check` subcommand: verifies the new skill is NOT in `ops/lib/manifests/CONTEXT.md` (Context Hazard enforcement).

2) **Update `AGENTS.md`:**
   - Add a directive to the **Contractor** role: "Upon closeout, Contractor shall propose

reusable skills via the Harvest Engine if a novel pattern or trap was encountered."

3) **Update `TASK.md`:**
  - Add a "Skill Harvest" checkbox to the `## 3) CLOSEOUT` section.

4) **Update `docs/library/MANUAL.md`:**
  - Add documentation for `ops/lib/skill/skill_lib.sh harvest` and `promote`.

5) **Self-Test (Bootstrap):**
  - Use the new script to harvest a "Lesson Learned" from *this* DP.
  - Create `S-LEARN-07: Automated Harvesting`.
  - Context: "Manual skill creation leads to drift and lack of provenance."
  - Solution: "Use the Harvest Engine to bind skills to DPs."

C) CHANGELOG
- **Log (human-readable, 1–6 bullets):**
 - Upgraded `ops/lib/skill/skill_lib.sh` to v2 (Harvest/Promote/Check).
 - Updated `AGENTS.md` to mandate skill harvesting considerations.
 - Updated `TASK.md` closeout checklist.
 - Documented new workflow in `MANUAL.md`.
 - Created `S-LEARN-07` as a proof-of-concept.

D) PATCH / DIFF
- **Format:** Unified diff (preferred) or anchored snippets.

E) RECEIPT (REQUIRED)

Verification (MUST RUN; or report NOT RUN + reason + risk)
- `./ops/bin/dump --scope=platform`
- `bash tools/context_lint.sh`
- `bash tools/lint_truth.sh`
- `bash tools/lint_library.sh`
- `bash tools/verify_tree.sh`

Skill capture (required during normal DP processing)
- Run `ops/lib/skill/skill_lib.sh promote` (using the draft generated in step 5).
- Include proof in RESULTS (grep output of `docs/library/INDEX.md` showing S-LEARN-07).

Artifact bundle (required directory)
- `storage/handoff/`

REQUIRED OUTPUT
MUST include a "Status" block at the top of RESULTS:

- `Scope summary:` (1 sentence validating the loop)
- `Tracked change:` (What was edited)
- `Verification:` (Output of proofs)

---

3) CLOSEOUT (MANDATORY)

Worker closeout duties (end of session)
- [ ] Produce/Update: `storage/handoff/DP-OPS-0011-RESULTS.md`
- [ ] If canon/governance surfaces were changed, include **SoP.md** in scope and update it in the same PR slice.
- [ ] Ensure proof bundle checklist is satisfied (above).
- [ ] Run `ops/lib/skill/skill_lib.sh check` to ensure no context hazards.
- [ ] Append a **TASK Work Log** entry (below) capturing:
  - what changed (high level)
  - gates/verification outcome (pass/fail or not run + reason)
  - blockers (if any)
  - **NEXT:** one single action

Integrator/operator closeout duties
- [ ] Review diff vs DP scope/forbidden zones
- [ ] Run repo-gates / required checks
- [ ] Commit/push/open PR/merge (operator-controlled)

Part VII: Technical Specification for `skill_lib.sh` v2

The following bash script implementation satisfies the requirements of `DP-OPS-0011` and the broader Harvesting Strategy. It is robust, secure, and adheres to the "Pointer-first" philosophy.

7.1 The Script Implementation

```bash
#!/usr/bin/env bash
set -euo pipefail

# Stela Skill Engine v2 (Harvest & Promote)
# Adheres to AGENTS.md: No contractions, quantitative reporting.
# Purpose: Transform ephemeral work into crystallized S-LEARN artifacts.
```

```
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE}")" && pwd)"
REPO_ROOT="$(cd "${SCRIPT_DIR}/../../.." && pwd)"
SKILL_LOG="${REPO_ROOT}/SKILL.md"
INDEX_FILE="${REPO_ROOT}/docs/library/INDEX.md"
CONTEXT_MANIFEST="${REPO_ROOT}/ops/lib/manifests/CONTEXT.md"
HANDOFF_DIR="${REPO_ROOT}/storage/handoff"
DRAFT_FILE="${HANDOFF_DIR}/DRAFT-SKILL.md"

mkdir -p "$HANDOFF_DIR"

usage() {
  cat <<'USAGE'
Usage: ops/lib/skill/skill_lib.sh [subcommand][args]

Subcommands:
  harvest   Analyze current state and draft a skill candidate.
  promote   Convert the active draft into a canonical S-LEARN file.
  check     Verify context safety (Negative Check).
USAGE
}

die() { echo "ERROR: $*" >&2; exit 1; }

# --- SUBCOMMAND: HARVEST ---
cmd_harvest() {
  echo "Initiating Harvest Mode..."

  # 1. Capture Provenance (Branch and Diff)
  local branch
  branch=$(git rev-parse --abbrev-ref HEAD)

  # Get a single line summary of changes
  local diff_stat
  diff_stat=$(git diff --stat main...HEAD | tail -n 1)

  if [[ -z "$diff_stat" ]]; then
    diff_stat="(No diff detected against main)"
  fi

  echo "Provenance: Branch=${branch}"
  echo "Diff Context: ${diff_stat}"

  # 2. Interactive Input (or Argument override)
```

```
  # We use simple read logic, assuming Operator or Agent driving via pipe.
  local name context solution

  if [[ -n "${1:-}" ]]; then
    name="$1"
    context="$2"
    solution="$3"
  else
    echo "Enter Skill Name (kebab-case, e.g., 'nextjs-caching'):"
    read -r name
    echo "Enter Context (The Problem/Trap):"
    read -r context
    echo "Enter Solution (The Pattern/Fix):"
    read -r solution
  fi

  # 3. Write Draft
  cat <<EOF > "$DRAFT_FILE"
# DRAFT CANDIDATE
Name: $name
Provenance: $branch
Context: $context
Solution: $solution
Diff_Stat: $diff_stat
EOF

  echo "Draft saved to: $DRAFT_FILE"
  echo "Review it, then run 'promote'."
}

# --- SUBCOMMAND: PROMOTE ---
cmd_promote() {
  if]; then
    die "No draft found at $DRAFT_FILE. Run 'harvest' first."
  fi

  echo "Initiating Promotion Mode..."

  # 1. Parse Draft (Simple extraction)
  local name context solution provenance
  name=$(grep "^Name:" "$DRAFT_FILE" | cut -d: -f2- | xargs)
  context=$(grep "^Context:" "$DRAFT_FILE" | cut -d: -f2- | xargs)
  solution=$(grep "^Solution:" "$DRAFT_FILE" | cut -d: -f2- | xargs)
```

```
  provenance=$(grep "^Provenance:" "$DRAFT_FILE" | cut -d: -f2- | xargs)

  # 2. Allocate ID (Collision Safe)
  local max_id=0
  # Scan disk for S-LEARN-XX to find the highest number
  while IFS= read -r file; do
    if+) ]]; then
      local num=$((10#${BASH_REMATCH}))
      if (( num > max_id )); then max_id=$num; fi
    fi
  done < <(find "${REPO_ROOT}/docs/library/skills" -name "S-LEARN-*.md")

  local next_id=$((max_id + 1))
  local skill_id
  skill_id=$(printf "S-LEARN-%02d" "$next_id")

  # 3. Context Hazard Check (Safety Rail)
  if grep -q "$skill_id" "$CONTEXT_MANIFEST"; then
    die "CONTEXT HAZARD: $skill_id is listed in CONTEXT.md. Skills must be on-demand only."
  fi

  # 4. Generate S-LEARN File
  local target_file="${REPO_ROOT}/docs/library/skills/${skill_id}.md"

  cat <<EOF > "$target_file"
# ${skill_id}: ${name}

## Provenance
Harvested from branch: ${provenance}
Date: $(date -u '+%Y-%m-%d')

## Scope
Production payloads only. Not platform maintenance.

## Context
${context}

## Invocation Guidance (The Fix)
${solution}

## Drift Preventers
- Stop if DP does not explicitly request this skill.
- Anti-hallucination: Verify paths against verify_tree.sh.
```

```
EOF

  echo "Created: $target_file"

  # 5. Register in INDEX.md (Library Guard)
  # Append to Section 5 using a robust check
  if! grep -q "$skill_id" "$INDEX_FILE"; then
    echo "skill-${name,,} | Skill: ${name} | docs/library/skills/${skill_id}.md" >> "$INDEX_FILE"
    echo "Registered in docs/library/INDEX.md"
  fi

  # 6. Log to SKILL.md
  local timestamp
  timestamp=$(date -u '+%Y-%m-%d %H:%M:%S UTC')
  echo "- ${timestamp} - ${skill_id} - ${name} (Harvested)" >> "$SKILL_LOG"

  # 7. Cleanup
  rm "$DRAFT_FILE"
  echo "Promotion Complete. Equilibrium restored."
}

# --- SUBCOMMAND: CHECK ---
cmd_check() {
  echo "Running Negative Context Check..."
  # Fails if any S-LEARN file is in the manifest
  # This prevents context window bloat and drift.
  if grep "S-LEARN" "$CONTEXT_MANIFEST"; then
    echo "FAIL: Skills detected in CONTEXT.md."
    exit 1
  fi
  echo "PASS: No skills in global context."
}

# --- MAIN DISPATCH ---
case "${1:-}" in
  harvest)
    shift
    cmd_harvest "$@"
    ;;
  promote)
    cmd_promote
    ;;
  check)
```

```
    cmd_check
    ;;
  *)
    usage
    exit 1
    ;;
esac
```

Part VIII: Conclusion and Future Implications

8.1 The Shift to "Self-Improving" Ops

The transition to an "Autonomous Skill Harvesting" model marks the maturation of the Stela platform from a static repository to a dynamic, self-improving operational system. By moving from manual, checklist-based skill creation to a provenance-backed, diff-aware harvesting engine, the system closes the loop between "doing work" (Fluid Intelligence) and "learning from work" (Crystallized Intelligence).

The proposed architectural changes—specifically the skill_lib.sh v2 and the strict negative checks in the Context Manifest—ensure that this new capability does not introduce "Drift" or bloat. The "Pointer-first" constitution remains the supreme law: if the skill isn't in the file, it doesn't exist; and if the skill wasn't harvested from a verified diff, it isn't a skill—it's just a hallucination.

8.2 Audit and Governance Impact

This upgrade significantly enhances auditability. Auditors can now trace every skill back to the specific incident (Provenance) that necessitated it. This transparency builds trust in the AI agents' logic. When an agent says, "I am applying S-LEARN-02," the operator knows this isn't a generic preference but a specific remediation for a known "Next.js Cache Storm" risk.

8.3 Final Recommendation

Execution of DP-OPS-0011 should proceed immediately following the verify_tree.sh confirmation of the current state. The re-writing of existing skills (S-LEARN-01 to 05) should be treated as a separate, subsequent DP (DP-OPS-0012: Retroactive Skill Harvesting) to maintain the "One Slice" rule of CONTRIBUTING.md. This ensures that the engine is built and tested (S-LEARN-07) before it is used to refactor the legacy library.

Feature	Current State (v1)	Future State (v2 - Harvest Engine)

Creation Method	Manual Append	Automated Harvest & Promotion
Provenance	None / Generic	Linked to Git Branch & Incident
Context Safety	Warning Text Only	Programmatic Negative Check
ID Allocation	Manual / Error Prone	Automated / Collision Safe
Governance	Disconnected from Diff	Driven by Diff Stats

The Stela platform is now poised to evolve from a "Managed" system to an "Adaptive" system, securing its place as a robust framework for autonomous agent operations.

Works cited

1. dump-full-main-b818b6193.txt
2. Agent Skills: The Universal Standard Transforming How AI Agents Work - Medium, accessed February 1, 2026, https://medium.com/@richardhightower/agent-skills-the-universal-standard-transforming-how-ai-agents-work-fc7397406e2e
3. Lessons learned - PMI, accessed February 1, 2026, https://www.pmi.org/learning/library/lessons-learned-next-level-Communicating-7991
4. Lessons learned guidelines & examples. How to automate them for project management, accessed February 1, 2026, https://www.clearpeople.com/blog/lessons-learned-guide
5. The Importance of Capturing Lessons Learned in Project Management - PMHut, accessed February 1, 2026, https://pmhut.com/the-importance-of-capturing-lessons-learned-in-project-management
6. License Harvesting: How to Detect Unused Software Licenses to Save Costs, accessed February 1, 2026, https://blog.invgate.com/software-reharvesting-cuts-costs-heres-how
7. OpenLM License Harvesting - Pay Less Utilize More, accessed February 1, 2026, https://www.openlm.com/blog/license-harvesting-openlm/
8. Automated License Harvesting: Reclaim Millions in Unused Software - Costif.ai Blog, accessed February 1, 2026, https://costif.ai/blog/license-harvesting-automation
9. Improving Latency of Database Calls in FastAPI with Asyncio, Lifespan Events, and

Connection Pooling - Usama Arif, accessed February 1, 2026, https://usamabjw.medium.com/improving-latency-of-database-calls-in-fastapi-with-asyncio-lifespan-events-and-connection-818066db59ab
10. Git Diff Explained: A Complete Guide with Examples - DataCamp, accessed February 1, 2026, https://www.datacamp.com/tutorial/git-diff-guide
11. Bash Heredoc: Complete Guide with Examples - Linuxize, accessed February 1, 2026, https://linuxize.com/post/bash-heredoc/
12. Heredocs in Bash Scripting - Anju - Medium, accessed February 1, 2026, https://anju-chaurasiya2012.medium.com/heredocs-in-bash-scripting-5f4d8b7589d1
13. 5 Lessons Learned From Taking Next.js App Router to Production - Inngest Blog, accessed February 1, 2026, https://www.inngest.com/blog/5-lessons-learned-from-taking-next-js-app-router-to-production
14. Migrating to the Next.js App Router (or: how I learned to stop worrying and love Server Actions) - Money Forward Developers Blog, accessed February 1, 2026, https://global.moneyforward-dev.jp/2025/12/04/migrating-to-the-next-js-app-router-or-how-i-learned-to-stop-worrying-and-love-server-actions/
15. Lifespan Events - FastAPI, accessed February 1, 2026, https://fastapi.tiangolo.com/advanced/events/